



SZABIST
UNIVERSITY
Islamabad Campus

Shaheed Zulfikar Ali Bhutto Institute of Science & Technology

COMPUTER SCIENCE DEPARTMENT

Total Marks: 04

Obtained Marks: _____

Parallel and Distributed Computing

Assignment # 03

Submitted To: Rana Faisal Hayat

Student Name: Ubaid Bin Waris

Reg Number: 2212416



COMPUTER SCIENCE DEPARTMENT

Instructions: *The written assignment must be completed on white pages. After writing, scan the assignment and upload it to Google Classroom before the due date. Copy-paste work or assignments with only changed names will not be accepted.*

Question No 1

A smart weather monitoring system uses message-oriented communication based on the publish–subscribe pattern. You must implement a simple weather-alert broadcasting service using ZeroMQ in Python.

The system consists of:

- A Weather Publisher (server) that periodically publishes temperature updates.
- A Weather Subscriber (client) that subscribes only to messages with topic "WEATHER" and displays the received temperature information.

I) Write Python code for a ZeroMQ Publisher that:

- Creates a PUB socket using a ZeroMQ context.
- Binds the socket to the address "tcp://HOST:PORT".
- Enters an infinite loop where, every 3 seconds, it sends a message whose topic is the literal string "WEATHER" and whose body contains the current city temperature (for example: "Islamabad 32°C").

II) Write Python code for a ZeroMQ Subscriber that:

- Creates a SUB socket using a ZeroMQ context.
- Connects the socket to the same "tcp://HOST:PORT" address as the publisher.
- Subscribes only to messages with the topic "WEATHER".
- Receives and prints 5 weather update messages from the publisher, then terminates.

Assume suitable values for HOST and PORT.

Question No 2

Consider the following two mpi4py code snippets for communication between process 0 and process 1 in a distributed image-processing system. One process generates image metadata, while the other process receives and processes it.

```
from mpi4py import MPI  
comm = MPI.COMM_WORLD
```

COMPUTER SCIENCE DEPARTMENT

```
rank = comm.Get_rank()
if rank == 0:
    image_info = {'width': 1920, 'height': 1080}
    comm.send(image_info, dest=1, tag=21)
elif rank == 1:
    image_info = comm.recv(source=0, tag=21)
```

```
from mpi4py import MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
if rank == 0:
    image_info = {'width': 1920, 'height': 1080}
    req = comm.isend(image_info, dest=1, tag=21)
    req.wait()
elif rank == 1:
    req = comm.irecv(source=0, tag=21)
    image_info = req.wait()
```

Briefly describe what each program does and identify which process sends and which process receives the image metadata.

- Explain the difference between blocking communication (send/recv) and nonblocking communication (isend/irecv) in MPI, using the above code as an example.
- In program (B), suppose the req.wait() call is removed from the sender side. What possible issue could occur during communication, and why?

COMPUTER SCIENCE DEPARTMENT

Question no. 1

Part 1

ZeroMQ Publisher

```
import zmq, time  
Context = zmq.Context()  
Socket = Context.Socket(zmq.PUB)  
  
Socket.bind("tcp://127.0.0.1:5000")
```

While True

```
    msg = "Weather Islamabad 32°C"  
    Socket.send_string(msg)  
    time.sleep(3)
```

Part 2

ZeroMQ Subscriber

```
import zmq  
Context = zmq.Context()  
Socket = Context.Socket(zmq.SUB)  
  
Socket.connect("tcp://127.0.0.1:5000")  
Socket.setsockopt_string(zmq.SUBSCRIBE, "Weather")
```

COMPUTER SCIENCE DEPARTMENT

```
for i in range(5)
    msg = socket.recv-string()
    Print(msg)
```

Question no. 2

```
comm.send(image_info, dest=1, tag=21)
```

Process 0 create image metadata

```
{'width': 1920, 'height': 1080}
```

Process 0 send this metadata to Process 1 using
send()

Process 1 receives the metadata using recv()

Sender and Receiver

Sender: Process 0

Receiver: Process 1

This use blocking communication.

COMPUTER SCIENCE DEPARTMENT

```
req = comm.isend(image_info, dest=1, tag=21)
```

Process 0 send image metadata using isend()

Process 1 receives data using irecv()

Both operations returns a request object (req)

req.wait ensure the communication is completed before continuing

Sender and Receiver

Sender : Process 0

Receiver : Process 1

This uses non blocking communication.

Question no. 3

Blocking Communication (send / recv)

Characteristics

- * The program wait until the communication operation finishes.
- * Execution stop until data transfer completes.

Advantages

- * simpler and easier to understand.
- * Good for small programs.

Disadvantages

- * Process may remain idle while waiting
- * low performance in parallel systems

COMPUTER SCIENCE DEPARTMENT

Non Blocking Communication (isend / irecv)

Characteristics

- * Communication starts immediately but the process can continue executing other tasks.
- * Communication is checked later using wait()

Advantages

- * Better parallelism and efficiency
- * Allow overlapping communication and computation.

Disadvantages

- * More complex to manage
- * Requires synchronization using wait() or test()

Question no. 4

If this line is removed

req. wait()

then the send operation may not complete properly before the sender continues or exits.

Possible issues

Data may not fully sent: The message transfer could still be in progress when the program terminates

COMPUTER SCIENCE DEPARTMENT

Communication Failure: The receiver may
receive incomplete data
receive corrupted data
or wait indefinitely for the message

Race condition:

Since `isend` is asynchronous, MPI only
initiates the communication.

Without `wait()` there is no guarantee that the
message was actually delivered.

Why `wait()` is important

the nonblocking operation completes successfully

The data is safely transmitted.

And synchronization occurs between processes

So, `wait()` is necessary to guarantee reliable communication in
non blocking MPI operations.